

Provisional Patent Application

Hardware Memory Protection System Using Base–Limit Register Pairs with Cryptographically Indistinguishable Invalid Access Responses

Inventor: Nick Spiker

Filing Date: [March 14, 2026]

Prior Art Established: January 1, 2025 (ferros architecture document)

ABSTRACT

A memory protection architecture for general-purpose operating systems providing process isolation thru per-process hardware base and limit registers without page tables or virtual memory translation, combined with a mechanism wherein invalid memory accesses return computationally indistinguishable responses derived from a hardware-protected secret key and the accessed address, preventing information disclosure about valid memory region boundaries by any sequence of memory access attempts.

The hardware key register output is physically wired to exactly one destination: the f computation logic. No path from the key register to any data bus exists in the circuit netlist. This is a structural property of the hardware, not a software-enforced policy. An attacker probing the key register's address receives cipher output per the invalid access response mechanism — a consequence of the address falling outside valid process bounds, independent of the key register's own wiring. The two protections are structurally independent and mutually reinforcing.

In a further embodiment, physical RAM is encrypted thruout using a stream cipher keyed by a hardware-protected key. This produces data and computational indistinguishability across three distributions: raw physical RAM reads, valid software memory reads, and invalid software memory reads are all computationally indistinguishable without knowledge of the hardware-protected keys. An attacker with complete software and physical access to bus signal values obtains no information about memory contents, valid address regions, or process memory boundaries.

In a further embodiment, both the valid memory data path and the f -derived response path execute unconditionally on every memory access, with a final multiplexer selecting the output. This extends indistinguishability to the physical side channel domain: power consumption and switching activity are identical for valid and invalid accesses, defeating power analysis attacks.

The address space is toroidal under two's complement arithmetic: no address is

structurally privileged, wraparound is handled correctly without special-case logic, and the null pointer dereference vulnerability class does not exist in this architecture.

FIELD OF THE INVENTION

Memory protection hardware and operating system architecture providing process isolation thru arithmetic bounds checking as a complete replacement for virtual memory and page table systems, combined with a cryptographically indistinguishable response mechanism for invalid memory accesses.

BACKGROUND

The Problem With Page Tables

All known general-purpose operating systems use virtual memory and page tables for process memory isolation. This approach requires:

- A Memory Management Unit (MMU) performing virtual-to-physical address translation on every memory access
- Translation Lookaside Buffer (TLB) hardware caching recent translations
- Page tables stored in RAM, requiring additional memory accesses on TLB miss
- Page-aligned allocations (typically 4 KB, 2 MB, or 1 GB granularity)
- Complex kernel code for page table management

This architecture has well-documented problems:

Performance. TLB miss incurs 4–5 memory reads (~ 100 –500 cycles). TLB hit requires 3–5 cycles minimum. Latency is non-deterministic, varying with working set size and TLB pressure.

Page granularity waste and unintended valid memory. Page table systems allocate memory in fixed-size pages (typically 4 KB). A process requesting a single byte receives a full 4 KB page: 4,095 bytes of valid, addressable memory the process did not request and may not have initialized. A process requesting 800 MB receives memory rounded up to the nearest page boundary. This over-allocation has two consequences. First, the bytes between the requested size and the page boundary are valid memory belonging to the process — adjacent allocations on the same page may read uninitialized data from neighboring allocations, a well-documented source of information disclosure vulnerabilities. Second, describing an 800 MB allocation requires approximately 204,800 page table entries ($800 \text{ MB} \div 4 \text{ KB}$), each consuming 8 bytes of kernel RAM, totaling $\sim 1.6 \text{ MB}$ of overhead just to describe one process's allocation. A system running many processes with large working sets carries correspondingly large page table structures consuming physical RAM and generating TLB pressure.

Alignment constraints. Page tables require base addresses aligned to page size. MMIO registers at arbitrary physical addresses require mapping entire pages, potentially

granting access to adjacent hardware registers belonging to different security domains.

Complexity. The Linux kernel contains millions of lines of page table management code, including demand paging, swap, huge pages, NUMA, OOM killer, and related subsystems — each a source of security vulnerabilities.

Information disclosure via deterministic fault responses. On all current operating systems, accessing memory outside a process's valid region produces a deterministic, observable signal: a page fault, SIGSEGV, bus error, or equivalent. This signal is itself information: the attacker learns that the probed address was outside valid memory. By systematically probing addresses and observing which produce faults and which succeed, an attacker can enumerate valid memory region boundaries. On x86-64, this binary search converges to 4 KB granularity — the page size — in $O(\log N)$ probes, where N is the address space size. With 2 MB huge pages the granularity is coarser but the attack remains practical. Address Space Layout Randomization (ASLR) raises the cost of this search but does not eliminate it: once a single fault-vs-success pair is observed, the search proceeds normally. Timing side channels [17] further reduce the cost of ASLR bypass. This invention eliminates the information disclosure problem at its root: invalid accesses produce cipher output indistinguishable from valid data, providing no signal that an invalid address was probed.

Prior Art and Its Limitations

Base and limit registers (1960s–1970s mainframes). Used in early systems including Cray supercomputers for memory partitioning. Required contiguous physical memory allocation. Replaced by virtual memory due to external fragmentation. No cryptographic response mechanism. Not used in modern general-purpose operating systems [1].

CHERI (Capability Hardware Enhanced RISC Instructions). University of Cambridge / SRI International, active development since 2010. Embeds base and bounds metadata in capability fat pointers (128-bit or 256-bit). Operates *alongside* conventional MMU and page tables as a hybrid model — explicitly not a replacement for virtual memory. Invalid capability access produces a hardware exception. Does not return cryptographically random or cipher-based data on invalid access. Requires page tables to remain operational [2, 3].

Mill Computing architecture. Protection Lookaside Buffer (PLB) separate from address translation, operating on virtual addresses. Still uses virtual memory and address translation. Invalid memory access produces NaR (Not a Result) metadata tag — a special sentinel value, not cryptographically random data. Does not make invalid access indistinguishable from valid access [4].

Intel MPX (Memory Protection Extensions). Bounds registers supplementing existing page-table-based protection. Deprecated in 2019. Invalid access produces #BR exception — not cryptographically random data [5].

ARM MTE (Memory Tagging Extension). Tag bits on memory allocations for temporal and spatial safety. Still requires MMU and page tables. Invalid tag access produces

exception — not random data [6].

ASLR (Address Space Layout Randomization). Randomizes base addresses of memory regions to make probing harder. Still uses page tables. Defeated by information leaks (Spectre, Meltdown, side channels). Provides probabilistic obscurity, not cryptographic guarantees [16, 17].

SUMMARY OF THE INVENTION

The invention provides a memory protection architecture comprising:

1. Per-process hardware base and limit register pairs replacing page tables as the sole memory isolation mechanism, with physical address equal to virtual address (MMU disabled or bypassed)
2. A two-operation bounds validation — subtract base, compare to limit — of deterministic, constant latency
3. For invalid accesses: a response derived from a hardware-protected secret key and the accessed address, such that the response is computationally indistinguishable from valid memory data without knowledge of the secret key
4. A hardware key register whose read-disable property is enforced by the same mechanism as invalid access response (Claim 2): any instruction that would read the register falls outside any valid process bounds and therefore receives the same cipher-derived output as any other invalid access, with no separate read path existing in hardware
5. A process table maintained by the kernel associating each running process with its base and limit registers; all address inputs to the response function are in the context of a process table entry and are natively hardware-word-width (usize), requiring no extension or truncation

Self-referential key protection. The hardware secret key is protected by the same mechanism it participates in. An attacker attempting to read the key register receives cipher-derived output indistinguishable from any other invalid access. The protection is uniform: there is no privileged path to key material at any software level, including kernel privilege. This property is intrinsic to the hardware wiring, not a software-enforced policy.

DETAILED DESCRIPTION

Independent Claim 1: Base–Limit Register Memory Protection (No Page Tables)

A memory protection system comprising:

- (a) At least one base register per process storing a physical base address
- (b) At least one limit register per process storing a maximum accessible offset from the base address
- (c) Hardware logic performing on every memory access:

```
offset = accessed_address - base_register
valid  = (offset < limit_register)    // unsigned comparison
```

- (d) No page table data structure required for process memory isolation
- (e) No TLB required for process memory isolation
- (f) Arbitrary base address alignment: base register may contain any address, not constrained to page boundaries
- (g) Arbitrary allocation size: limit register may specify any byte count, not constrained to power-of-two page sizes
- (h) Physical addresses equal virtual addresses; the MMU is disabled or bypassed entirely, eliminating address translation overhead
- (i) A kernel-maintained process table associating each running process with one or more base–limit register pairs; physical memory assignment is managed directly by the kernel without page table data structures

Distinction from prior art. Classic base/limit systems [1] required page-aligned contiguous physical memory and were abandoned due to fragmentation. This invention operates in a modern general-purpose OS context with no page tables, no MMU translation, and no alignment constraints. CHERI [2] uses similar bounds checking but retains page tables and MMU as required components and does not disable virtual memory.

Independent Claim 2: Cryptographically Indistinguishable Invalid Access Response

A memory protection system wherein invalid memory accesses return data computed as:

$$response = f(secret_key, accessed_address)$$

where:

- (a) *secret_key* is stored in a hardware register enforced read-disabled as described in Claim 4
- (b) *accessed_address* is a hardware-word-width (usize) value drawn from the process table context; no extension or truncation of the address is required
- (c) *f* is a deterministic function such that:
 - The same address always produces the same response for a given key
 - The response is computationally indistinguishable from valid memory contents without knowledge of *secret_key*
 - The response is computed in hardware
- (d) No error signal, exception, fault, or otherwise distinguishable response is generated for invalid accesses

Distinction from prior art:

- Page fault / SIGSEGV (all conventional OS): deterministic error, reveals boundary location

- NaR (Mill Computing [4]): special sentinel value, distinguishable from valid data by inspecting metadata
- Hardware exception (CHERI [2], MPX [5], MTE [6]): deterministic error signal
- Zero / fixed value: distinguishable from valid data by inspection
- Bus error / hang: distinguishable by timing

This invention is the first to provide computationally indistinguishable responses for invalid memory accesses.

Dependent Claim 3: Double-Read Attack Resistance (depends on Claim 2)

A memory protection system according to Claim 2 wherein the deterministic property of f prevents double-read distinguisher attacks:

- (a) An attacker reading an invalid address twice receives identical values
- (b) Valid memory and invalid memory are indistinguishable by repeated access patterns
- (c) Achieved by: $f(secret_key, address)$ being deterministic for a given (key, address) pair while being computationally random without knowledge of $secret_key$

Dependent Claim 4: Hardware Key Protection (depends on Claim 2)

A memory protection system according to Claim 2 wherein:

- (a) $secret_key$ is stored in a hardware register whose output is physically wired to exactly one destination: the input of the f computation logic (the ARX compression function, block cipher, or equivalent per Claim 7). No other wire, bus connection, or signal path from the key register output exists in the hardware implementation. This is a property of the circuit netlist, not of software policy or privilege gating.
- (b) Specifically: the key register output is not connected to the processor data bus, not connected to any multiplexer whose other input reaches the data bus, and not readable by any instruction in the ISA. The absence of a read path is structural — there is no gate to bypass, no privilege check to circumvent, no microcode path to exploit. The key material cannot appear on the data bus because no wire connecting the key register to the data bus exists.
- (c) The read-disable property is therefore not circular with respect to Claim 2. Claim 2 describes the response produced when a software access falls outside valid bounds. Claim 4 describes a separate physical constraint: the key register output wire terminates at the f logic and nowhere else. An attacker probing the key register's address receives $f(secret_key, address)$ per Claim 2 because that address is outside any valid process region — not because the key register has a special protection mechanism. The two properties are independent and mutually reinforcing, not circularly dependent.
- (d) The register accepts writes (to install a new key via a privileged instruction) but the write path is one-way into the register storage element only; the output of the storage element is wired exclusively to the f logic as described in (a).

- (e) The key is derived fresh at each boot:

$$key = cipher(device_key, boot_nonce)$$

where *device_key* is a per-device secret stored in one-time-programmable (OTP) fuses or equivalent non-volatile hardware storage, similarly having no software-readable output path.

- (f) Key erasure: power removal causes immediate loss of *secret_key* (volatile register storage). *device_key* persists in OTP but is not sufficient to reconstruct *secret_key* without the *boot_nonce* consumed at boot time.

Result. An attacker with full software access including kernel privilege cannot extract *secret_key* because no instruction causes the processor to place the register contents on any data bus — there is no such path in the circuit. An attacker probing the key register's memory address receives cipher output per Claim 2 because that address is outside all valid process regions. The two protections are structurally independent; neither depends on the other for its validity.

Dependent Claim 5: MMIO Grant Arbitrary Granularity (depends on Claim 1)

A memory protection system according to Claim 1 wherein hardware access grants for memory-mapped I/O (MMIO) specify:

- (a) Arbitrary base address (not page-aligned)
- (b) Arbitrary limit (byte granularity)

Result. A process granted access to a specific hardware register at address X with limit N bytes cannot access address $X - 1$ or $X + N$, regardless of page boundaries. This eliminates the page granularity problem wherein granting access to one hardware register inadvertently grants access to adjacent registers sharing the same page.

Dependent Claim 6: Optional Address Anonymization (depends on Claim 1)

A memory protection system according to Claim 1 wherein valid memory accesses optionally add a per-process random offset:

$$physical = base + offset + \delta$$

where δ is a process-specific value derived from *secret_key*, providing additional address space anonymization.

Note: This mechanism is separable from and not required by Claims 1–5. It may operate at varying frequencies determined by the kernel, including continuous rotation, without affecting the validity of the bounds check in Claim 1. Whether a process knows or does not know its own absolute physical address is irrelevant to the security property of Claim 2: denied access is denied access regardless of address knowledge.

Dependent Claim 7: Embodiments of the Invalid Access Response Function (depends on Claim 2)

The function $f(\text{secret_key}, \text{address})$ of Claim 2 may be implemented as any of the following, all providing the computational indistinguishability property of Claim 2 under the assumption that secret_key is not extractable:

- (a) **Reduced-round block-cipher-derived compression function.** One or more rounds of a BLAKE3-derived or similar ARX (add-rotate-XOR) compression function, keyed with secret_key , applied to address as message input. The number of rounds may be reduced relative to full specification counts; under hardware key non-extractability, a single round of the compression function is sufficient for computational indistinguishability, as distinguishing reduced-round ARX output from random requires knowledge of the key. Full-specification round counts constitute a higher-security embodiment.
- (b) **Block cipher.** $\text{AES}(\text{secret_key}, \text{address_block})$, single-block evaluation. Hardware AES acceleration available on ARM (AES instructions) and custom silicon.
- (c) **Stream cipher output.** $\text{ChaCha20}(\text{secret_key}, \text{address})$ evaluated for the relevant byte range.
- (d) **Keyed hash.** $\text{BLAKE3}(\text{secret_key} \parallel \text{address})$ truncated to access width.
- (e) **XOR with key material.** $\text{address} \oplus \text{key_slice}(\text{address})$. Computationally simple; acceptable when secret_key is not extractable by any hardware or software path.
- (f) **Polynomial evaluation over $\text{GF}(2^{128})$.** $\text{response} = \text{key} \cdot \text{address} + \text{key}^2$ evaluated over a Galois field. Equivalent to the GHASH function used in AES-GCM. Hardware implementations exist. Latency is deterministic and low.

Dependent Claim 8: Timing Indistinguishability (depends on Claim 2)

A memory protection system according to Claim 2 wherein:

- (a) The response function $f(\text{secret_key}, \text{address})$ is computed in hardware on every memory access, in parallel with the bounds check of Claim 1, regardless of whether the access is ultimately valid or invalid
- (b) A multiplexer selects between the valid memory data path and the f -derived response based on the bounds check result; both paths complete before the multiplexer output is emitted
- (c) The latency of the invalid access response is not observably faster or slower than the latency distribution of valid memory accesses across realistic workloads, which include L1, L2, L3, and DRAM access latencies spanning multiple orders of magnitude
- (d) No timing side channel distinguishes valid from invalid accesses in a statistical sense: invalid-access response latency falls within the natural variance of valid-access latency observed by an attacker with no knowledge of the memory hierarchy state

- (e) Specifically: an implementation using a reduced-round ARX compression function per Claim 7(a) produces invalid-access response latency consistent with L2 cache miss latency, which is indistinguishable from valid L2 misses in any observable access pattern

Distinction from prior art. All conventional invalid access responses (page fault, exception, hang, NaR) have response latencies dramatically different from valid access latencies, enabling timing-based boundary detection [17]. This invention produces no such distinguishable latency signal.

Dependent Claim 9: RISC-V Custom Extension Embodiment (depends on Claim 1)

The base and limit registers of Claim 1 implemented as:

- (a) Custom Control and Status Registers (CSRs) in the RISC-V ISA
- (b) Per-hart (hardware thread) storage
- (c) Accessible only in supervisor/machine mode
- (d) Combined with invalid access response hardware (Claim 2) in the memory access pipeline

Dependent Claim 10: ARM System Register Embodiment (depends on Claim 1)

The base and limit registers of Claim 1 implemented using:

- (a) ARM system registers (accessible via MRS/MSR instructions)
- (b) MMU disabled (`SCTLR_EL1.M = 0`); this is a supported hardware configuration with fully documented system register semantics on all ARMv8-A and ARMv9-A implementations; physical addresses equal virtual addresses
- (c) The bounds check of Claim 1 replaces all address translation formerly performed by the MMU; no TF-A (Trusted Firmware-A), hypervisor, or other firmware layer performing MMU management is required or beneficial in this configuration
- (d) ARM RNDR instruction as entropy source for Claim 7(e) embodiment; RNDR is compliant with NIST SP 800-90B [14] and constitutes a hardware true random number generator per definition, as deterministic algorithms cannot pass NIST 800-90B entropy validation tests regardless of vendor labeling [13]
- (e) Combined with Claim 17: ARM context transitions require no TLBI, no DSB/ISB barriers, and no `MSR_TTBRO_EL1` instruction, because no translations exist to invalidate; the ARM pipeline executes the new process's first instruction in the cycle following the *current_pid* register update with no stall

Independent Claim 11: Combination Claim (incorporates Claims 1–4)

A memory protection system combining:

- (a) Base–limit register process isolation, no page tables (Claim 1)

- (b) Invalid access: $f(secret_key, address)$, deterministic per address (Claim 2)
- (c) $secret_key$: hardware read-disabled register, protection enforced by the same mechanism as (b) with no separate read path in hardware (Claim 4)
- (d) Double-read resistant: same address always yields same response (Claim 3)
- (e) Timing indistinguishable: invalid access response latency statistically indistinguishable from valid access latency (Claim 8)
- (f) No information about valid address regions, memory contents, or process boundaries is disclosed by any sequence of memory access attempts

Dependent Claim 12: Encrypted Physical Memory Embodiment (depends on Claim 11)

A memory protection system according to Claim 11 wherein:

- (a) Physical RAM contents are encrypted using a stream cipher (ChaCha20 or equivalent) keyed with a hardware-protected key
- (b) Encryption and decryption occurs transparently in the memory controller or DMA path, not in the CPU pipeline
- (c) The bounds check of Claim 1 operates on unencrypted addresses using register arithmetic, requiring no access to encrypted RAM contents
- (d) The invalid access response of Claim 2 is generated without accessing physical RAM — derived entirely from $f(secret_key, address)$
- (e) Every read from any address at any level returns cipher-like output:
 - Raw RAM read (physical attack): returns ciphertext
 - Valid software read: returns decrypted data
 - Invalid software read: returns $f(secret_key, address)$

All three are computationally indistinguishable without knowledge of the hardware-protected keys
- (f) Physical memory extraction yields only ciphertext providing no information about process memory contents, valid/invalid region boundaries, or address space layout

Distinction from prior art. Full RAM encryption implementations (AMD SME/SEV [9], Intel TME [10]) operate alongside page tables, which are themselves stored in encrypted RAM, requiring complex bootstrap sequences to decrypt page table entries before address translation can proceed. This invention contains no page tables in RAM. The bounds check is pure register arithmetic. Encryption is an independent layer with no bootstrap dependency. The two mechanisms are formally separable and independently provable.

Dependent Claim 13: Data and Computational Indistinguishability (depends on Claims 11 and 12)

A memory protection system combining Claims 2, 8, and 12 wherein, with respect to *data content and computational distinguishers*:

$\forall$ attacker A without hardware-protected keys:

$$\text{dist}(\text{valid_access_response}) \equiv_c \text{dist}(\text{invalid_access_response}) \equiv_c \text{dist}(\text{raw_RAM_contents})$$

where $\equiv_c$ denotes computational indistinguishability: no probabilistic polynomial-time algorithm distinguishes the distributions with non-negligible advantage.

Scope of this claim. This claim asserts indistinguishability with respect to:

- Data values returned by memory read operations
- Data values observable on the memory bus
- Computational analysis of any sequence of access results

This claim does not assert indistinguishability with respect to physical side channels (power consumption, electromagnetic emission, photonic emission, or similar) in implementations that do not also implement Claim 16. Claim 16 provides the constant-power embodiment that extends indistinguishability to the physical side channel domain.

Proof basis. All three distributions are outputs of keyed cipher operations under keys that are hardware-protected and have no software-readable output path per Claim 4. Security reduces to the semantic security of the underlying cipher: any distinguisher for the memory system distributions yields a distinguisher for the cipher, which contradicts the cipher's security assumption.

An attacker with complete software access including kernel privilege, and complete physical access to RAM bus signal values, obtains no information about:

- Which memory addresses are valid for any process
- What data any process contains
- Where any process's memory region boundaries lie

Dependent Claim 16: Constant-Power Physical Side Channel Resistance (depends on Claim 13)

A memory protection system according to Claim 13 wherein:

- (a) The f computation logic of Claim 2 and the valid memory data path of Claim 8 operate simultaneously on every memory access, regardless of whether the access is valid or invalid, such that both logic paths consume power on every cycle
- (b) The multiplexer selecting between the valid data path and the f -derived response is the last logic element before output; all preceding computation is unconditional
- (c) Power consumption, electromagnetic emission, and switching activity of the memory protection hardware are therefore identical for valid and invalid accesses: both paths are fully evaluated on every access
- (d) No power analysis (Simple Power Analysis, Differential Power Analysis, or equivalent) performed on the memory protection hardware distinguishes valid from invalid memory accesses, because the hardware performs identical computation in both cases

- (e) Combined with Claim 13, this embodiment extends indistinguishability from the data and computational domain to the physical side channel domain

Distinction from prior art. Conventional memory protection hardware (page fault, exception, bounds check fault) performs qualitatively different operations on valid versus invalid accesses, producing distinguishable power signatures. ARM MTE [6], CHERI [2], and Intel MPX [5] all generate exceptions on invalid access, consuming power patterns distinct from valid access paths. This invention evaluates both paths unconditionally on every access, making the power trace uninformative about access validity.

Dependent Claim 14: Uniform Address Space — No Privileged Positions (depends on Claims 1–3)

A memory protection system according to Claims 1–3 wherein:

- (a) Every address in the hardware address space is structurally equivalent: no address is reserved, privileged, or treated specially by the memory protection hardware on the basis of its numeric value alone
- (b) Memory allocation may begin at any address and extend for any byte count, including allocations whose address range crosses the arithmetic wraparound boundary of the address space
- (c) The bounds check of Claim 1 — computed as $(\text{address} - \text{base_register})$, result compared unsigned against limit_register — correctly validates all accesses including those crossing the wraparound boundary, without special-case logic. To make this concrete: unsigned subtraction in two's complement hardware naturally produces a large positive result when an address below base is subtracted from base , because the subtraction wraps around to a massive value that always exceeds any legitimate limit . The same single comparison therefore validates both the lower bound (address not below base) and the upper bound (address not above $\text{base} + \text{limit}$), with wraparound handled correctly as a mathematical consequence of unsigned arithmetic rather than a special case.

Note on naive implementation. The intuitively obvious alternative — two separate comparisons:

```
// WRONG: does not form a ring
if (address >= base && address < base + limit) { valid }
```

produces a linear region with two hard endpoints. It does not handle allocations that cross the wraparound boundary: an allocation beginning near the top of the address space and extending past $2^N - 1$ back toward zero will be incorrectly rejected or require an explicit branch to detect and handle the wraparound case. Adding that branch is simply implementing unsigned modular subtraction the hard way, at the cost of extra logic and a conditional that the subtract-compare eliminates entirely. The correct single-instruction form is:

```
// CORRECT: forms a ring, handles wraparound by construction
```

```
if ((address - base) < limit) { valid } // all unsigned
```

where all values are treated as unsigned integers and subtraction is allowed to wrap. This is not a micro-optimization — it is the difference between a system with two detectable endpoints and a uniform address space with none

- (d) An allocation of size L beginning at base B occupies exactly the addresses B through $(B + L - 1) \bmod 2^N$ where N is the address width; no rounding to page boundaries occurs; the process receives exactly the bytes requested and no more
- (e) Address zero is not reserved, trapped, or treated differently from any other address by the memory protection hardware; it is a valid address within any region whose base–limit pair contains it, and an invalid address returning $f(\text{secret_key}, 0)$ per Claim 2 in any region that does not contain it — identical behavior to every other address
- (f) As a consequence of (e): the null pointer dereference vulnerability class (CWE-476 and related) does not exist in this architecture. On all current systems, address zero is unmapped by convention so that a null pointer dereference produces a deterministic fault that developers rely upon for bug detection. This convention is unnecessary in this architecture because the memory protection hardware already handles out-of-bounds accesses uniformly at all addresses; no address requires special-case unmapping to produce safe behavior

Distinction from prior art:

- *All page-table systems (Linux, Windows, macOS, etc.):* address zero is unmapped by OS convention, producing a detectable fault on access (SIGSEGV, access violation). The entire null pointer dereference CWE class exists because this convention is relied upon rather than designed away.
- *x86 segmentation:* segments have defined numeric start and end; access past the segment limit produces a general protection fault (#GP); wraparound across the segment limit is not supported and produces a fault.
- *CHERI [2]:* the null capability (all-zero capability value) is a defined architectural special case; capability errors corresponding to null pointer dereference remain present as a distinct architectural condition.
- *This invention:* no address is architecturally special; wraparound is correct by construction via unsigned arithmetic; null pointer dereference as a vulnerability class does not exist because no address is reserved to produce a special fault response.

Dependent Claim 15: Write Indistinguishability (depends on Claims 1–3)

A memory protection system according to Claims 1–3 wherein:

- (a) Invalid writes — writes to addresses outside the process’s granted base–limit region — complete silently in hardware: data is discarded with no fault, exception, error signal, or timing difference observable to the writing process

- (b) A process may determine whether an address falls within its own granted region by write-then-read-back; this reveals no information beyond what the process was explicitly granted by the kernel at region allocation time and constitutes no security violation
- (c) Cross-process boundary probing by write-then-read-back is prevented: a process holds no write grant to any address outside its own base–limit region; any read of addresses outside that region returns $f(secret_key, address)$ per Claim 2 regardless of whether a write was attempted, because the write itself was discarded
- (d) The cipher response mechanism of Claim 2 is therefore the operative protection against cross-process boundary probing, against physical and bus-level observation, and against external timing analysis; Claim 15 establishes that the write path provides no additional oracle beyond what the read path already closes
- (e) Specifically: the cryptographically uniform response of Claim 2 is necessary and sufficient for cross-process indistinguishability; silent write discard ensures no write-path side channel exists in addition to the read-path protection

Distinction from prior art:

- *POSIX mmap with separate PROT_READ / PROT_WRITE*: architecturally separable read and write permissions permit write-then-read-back cross-region probing under some configurations
- *CHERI [2]*: separate read and write permission bits on capabilities; write-without-read grants are architecturally possible
- *This invention*: write outside granted region is silently discarded; read outside granted region returns cipher; no cross-process information is obtainable by any combination of read and write operations

Dependent Claim 17: Hardware Process Table with Zero-Stall Context Transition (depends on Claim 1)

A memory protection system according to Claim 1 wherein:

- (a) The base and limit register pairs for all N concurrently managed processes are stored in a fully-resident hardware register file — a process table — such that every process's base and limit values are present in hardware at all times and require no load operation before use
- (b) The active base and limit values presented to the bounds check logic of Claim 1 are the combinatorial outputs of a multiplexer indexed by a current-process index register (*current_pid*); no register stage exists between the process table output and the bounds check inputs, such that the propagation delay of the multiplexer is absorbed into the existing combinatorial delay budget of the bounds check and does not extend the pipeline critical path
- (c) The scheduler — whether implemented in hardware or as a privileged kernel operation — maintains a *next_pid* value computed one clock edge ahead of the required transition, such that when *current_pid* is updated the combinatorial

bounds check logic immediately presents the new process's base and limit values with no pipeline bubble

- (d) A context transition consists of a single register update:

```
current_pid <= next_pid;  // one clock edge, no stall cycles
```

No TLB flush, no pipeline flush, no memory barrier, no ASID update, no page table pointer swap, and no broadcast to other cores is required, because no translations exist to invalidate

- (e) In-flight memory accesses carry their bounds check result (*valid*) as a pipeline register from the point of evaluation forward; the validity of an in-flight access is determined by the process that issued it and is not re-evaluated if *current_pid* changes while the access is in flight, eliminating race conditions at context transition boundaries:

```
// Bounds check result registered with the request
always @(posedge clk)
    valid_r <= (mem_addr - active_base) < active_limit;
```

- (f) On multi-core implementations: each core maintains its own *current_pid* register; the process table register file is shared read-only hardware state visible to all cores; writes to the process table (kernel installing or updating a process's base/limit) require arbitration on the write port only and do not affect the read/execute path of any core
- (g) The result: a context transition imposes zero pipeline stall cycles. The transition is invisible to pipeline throughput. The process table lookup is on the combinatorial critical path of the bounds check, which is itself the critical path of every memory access; the process table mux propagation delay must therefore be less than or equal to the propagation delay of the subtract and compare operations of Claim 1 for the zero-stall property to hold. This is achievable in standard cell implementations for process counts up to several hundred entries before mux depth begins to affect timing.

Distinction from prior art:

- *x86-64 context switch*: requires CR3 load (new page table base), TLB invalidation or ASID management, pipeline flush, and memory barriers (DSB/ISB on ARM equivalent); cost is tens to hundreds of cycles depending on TLB state and core count
- *ARM context switch with MMU*: requires TLBI ASIDE1IS (TLB invalidate by ASID, inner shareable domain broadcast to all cores), DSB ISH before and after, ISB to flush pipeline, MSR TTBR0_EL1 to install new page table; minimum six instructions, several stalling the pipeline on memory system acknowledgment; cost scales with core count due to TLB broadcast
- *CHERI*: operates alongside page tables; context switch retains full page-table-based overhead plus additional capability register state save/restore

- *seL4 microkernel* [7]: achieves world-record context switch times of ~ 300 cycles on ARM by careful optimization of the page-table-based switch path; this invention eliminates the page-table overhead that makes such optimization necessary, reducing the irreducible minimum to the single clock edge of the *current_pid* register update
- *This invention*: context transition cost is one clock edge; zero pipeline stall cycles; no TLB, no barrier, no broadcast; cost does not scale with core count; the concept of context switch overhead as a design constraint effectively ceases to exist

PRIOR ART CITE LIST

The following prior art is disclosed. The invention is distinguished from each item as noted in the claims above.

References

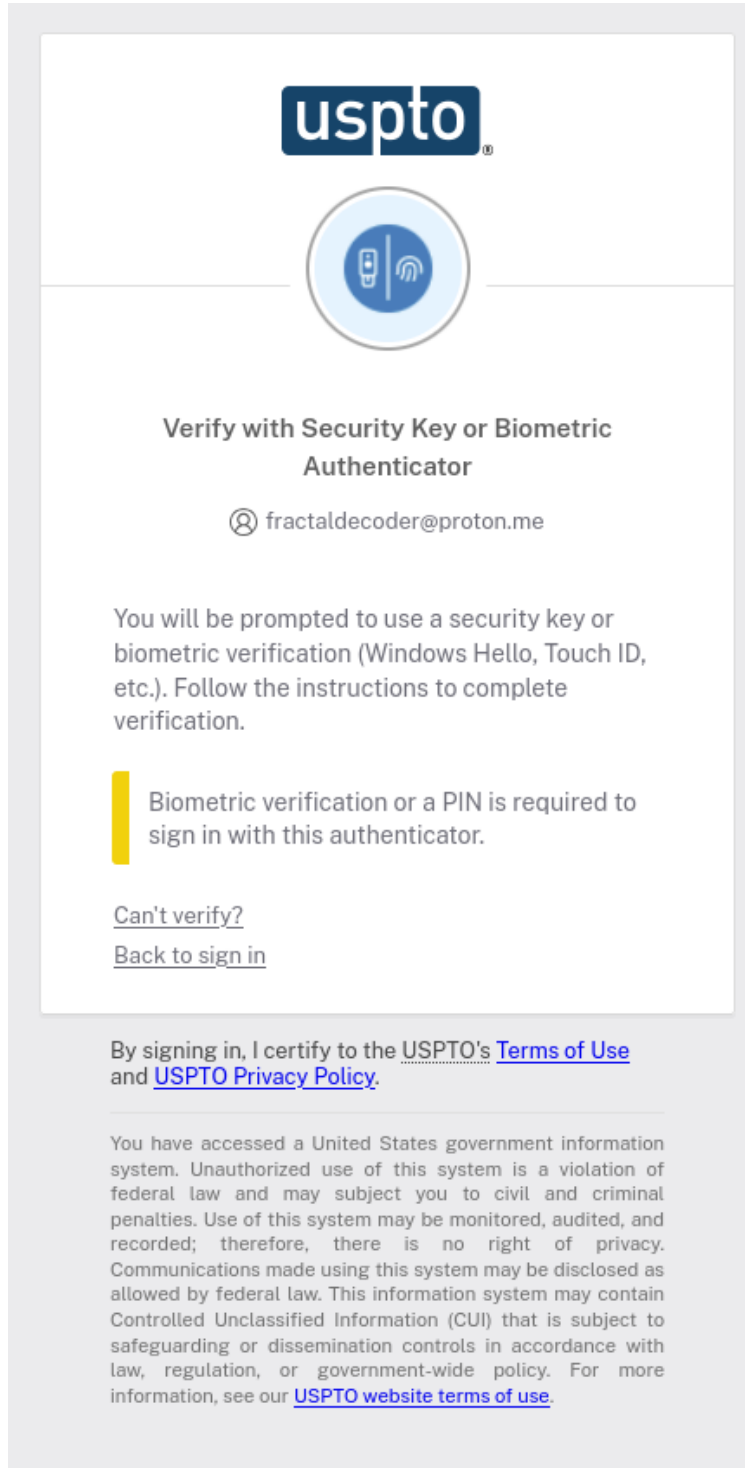
- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed., Chapter 9: Memory Management. Base and limit registers for contiguous memory allocation. *Distinguished*: requires contiguous physical memory and page alignment; replaced by virtual memory due to fragmentation.
- [2] R. N. M. Watson et al., “CHERI: A Hybrid Capability-System Architecture for Scalable Software Compartmentalization,” *Proc. ISCA*, 2015. *Distinguished*: hybrid model retaining page tables and MMU; invalid access produces hardware exception, not cipher output.
- [3] R. N. M. Watson et al., *Capability Hardware Enhanced RISC Instructions (CHERI) ISA Specification*, University of Cambridge Technical Report. *Distinguished*: as above.
- [4] Mill Computing, Inc., “Protection,” millcomputing.com/wiki/Protection. *Distinguished*: retains virtual memory; PLB not a page table replacement; invalid access returns NaR metadata, not cipher output.
- [5] Intel Corporation, *Intel Memory Protection Extensions*, Software Developer Manual. *Distinguished*: supplemental to page tables; invalid access produces #BR exception.
- [6] ARM Limited, *ARM Memory Tagging Extension Architecture Specification*. *Distinguished*: supplemental to page tables; invalid access produces exception.
- [7] G. Klein et al., “seL4: Formal Verification of an Operating-System Kernel,” *Commun. ACM*, 2010. *Distinguished*: uses page tables, ASID, MMU.
- [8] D. Patterson and J. Hennessy, *Computer Organization and Design: RISC-V Edition*, Chapter on virtual memory and TLBs. *Distinguished*: describes page table architecture this invention replaces.

- [9] AMD Corporation, *AMD Secure Memory Encryption (SME) and Secure Encrypted Virtualization (SEV) Architecture Guide*. *Distinguished*: operates alongside page tables stored in encrypted RAM; requires complex bootstrap to decrypt page table entries before address translation can proceed. This invention contains no page tables in RAM.
- [10] Intel Corporation, *Intel Total Memory Encryption (TME) Architecture Specification*. *Distinguished*: supplemental to existing page-table-based virtual memory; does not address invalid access response indistinguishability.
- [11] ARM Limited, *Realm Management Extension (RME)*. *Distinguished*: confidential computing extension operating alongside standard MMU and page tables.
- [12] Cryptography Research Inc., “Analysis of Intel’s Ivy Bridge Digital Random Number Generator,” 2012.
- [13] ARM Limited, *ARM Architecture Reference Manual, ARMv8*, Section on RNDR/RNDRRS instructions. *Note*: ARM labels RNDR “pseudo-random” but NIST SP 800-90B compliance establishes it as a true random number generator per definition; deterministic algorithms cannot pass 800-90B entropy validation tests.
- [14] NIST SP 800-90B, *Recommendation for the Entropy Sources Used for Random Bit Generation*, 2018.
- [15] NIST SP 800-90A, *Recommendation for Random Number Generation Using Deterministic Random Bit Generators*, 2015.
- [16] PaX Team, “PaX ASLR (Address Space Layout Randomization),” 2001. *Distinguished*: randomizes region base addresses; does not affect response to invalid accesses; defeated by information leaks.
- [17] R. Hund, C. Willems, and T. Holz, “Practical Timing Side Channel Attacks Against Kernel Space ASLR,” *IEEE S&P*, 2013. Demonstrates ASLR defeat via timing — problem this invention resolves.
- [18] J. B. Dennis and E. C. Van Horn, “Programming Semantics for Multiprogrammed Computations,” *Commun. ACM*, 1966. Original capability model. *Distinguished*: software capabilities, not hardware memory protection bounds checking.
- [19] H. M. Levy, *Capability-Based Computer Systems*, 1984. Historical survey. *Distinguished*: early capability systems used indirection tables, not arithmetic bounds checking.

EXHIBIT A: USPTO FILING SYSTEM UNAVAILABILITY

On 2026-03-14, the USPTO electronic filing system required security key or biometric authentication that could not be completed, preventing electronic submission of this provisional application. This screenshot documents the system state at the time the inventor was prepared to file.

Provisional Patent Application
Inventor: Nick Spiker
All rights reserved



The image shows a screenshot of the USPTO login portal. At the top is the USPTO logo. Below it is a circular icon containing a security key and a fingerprint symbol. The main heading reads "Verify with Security Key or Biometric Authenticator". Below this is the email address "fractaldecoder@proton.me" preceded by a person icon. A paragraph explains that the user will be prompted to use a security key or biometric verification (Windows Hello, Touch ID, etc.) and to follow the instructions to complete verification. A yellow bar highlights a message: "Biometric verification or a PIN is required to sign in with this authenticator." Below this are two links: "Can't verify?" and "Back to sign in". At the bottom, there is a certification statement: "By signing in, I certify to the USPTO's Terms of Use and USPTO Privacy Policy." followed by a detailed disclaimer about the system being a United States government information system with various legal notices and a link to the USPTO website terms of use.

uspto



Verify with Security Key or Biometric Authenticator

 fractaldecoder@proton.me

You will be prompted to use a security key or biometric verification (Windows Hello, Touch ID, etc.). Follow the instructions to complete verification.

Biometric verification or a PIN is required to sign in with this authenticator.

[Can't verify?](#)
[Back to sign in](#)

By signing in, I certify to the USPTO's [Terms of Use](#) and [USPTO Privacy Policy](#).

You have accessed a United States government information system. Unauthorized use of this system is a violation of federal law and may subject you to civil and criminal penalties. Use of this system may be monitored, audited, and recorded; therefore, there is no right of privacy. Communications made using this system may be disclosed as allowed by federal law. This information system may contain Controlled Unclassified Information (CUI) that is subject to safeguarding or dissemination controls in accordance with law, regulation, or government-wide policy. For more information, see our [USPTO website terms of use](#).

Figure 1: USPTO login portal on 2026-03-14, blocking access with mandatory security key / biometric verification.